

EE/CS 148B HW1: Build a Language Model from Scratch

Alex Chung

May 2026

Contents

1	Tokenizer	2
1.1	Insight (BPE): Naive Merging	2
1.2	BPE Training on TinyStories	2
2	Transformer Architecture	2
3	Training	2
3.1	Training Setup	2
3.2	Reaching Validation Loss ≤ 2.0	3
4	Text Generation	3
4.1	Sampling Implementation	3
4.2	Insight: Temperature Scaling	4
4.3	Generated Text Sample	4
5	Insight: Perplexity	5
6	Ablation 1: Effect of LayerNorm	5
6.1	Setup	5
6.2	Observations and Explanation	5
7	Ablation 2: Sinusoidal PE vs. NoPE	6
7.1	Setup	6
7.2	Observations and Explanation	6
8	Summary of Results	7

1 Tokenizer

1.1 Insight (BPE): Naive Merging

Naively merging pairs of bytes across raw text can create separate tokens for words that are semantically almost identical but differ only in punctuation. For example, `dog!` and `dog?` could become completely different tokens, even though both contain the same word `dog` and should share much of their meaning. This matters because downstream models must then learn that `dog!` and `dog?` are related concepts from context alone, wasting model capacity that could have been used for genuinely distinct vocabulary.

1.2 BPE Training on TinyStories

Training the byte-level BPE tokenizer on TinyStories completed in 6.7 minutes, which is well within the 30-minute limit. The final tokenizer had 10,000 tokens total, consisting of 256 initial byte tokens, 1 special token (`<|endoftext|>`), and 9,743 learned merges.

The longest token in the vocabulary was `accomplishment`, which is 15 bytes long. This result makes sense because `accomplishment` is a long English word that appears often enough in the TinyStories dataset for BPE to merge its bytes into a single token. The leading space is also expected because the GPT-2-style pre-tokenizer treats space-prefixed words as units, so the tokenizer correctly learns frequent subword patterns in their natural word-boundary context.

2 Transformer Architecture

The model is a decoder-only Transformer language model built entirely from primitive operations (no `torch.nn.functional` calls). Key components:

- **Embedding:** token embedding lookup + sinusoidal positional embedding (fixed, non-learnable).
- **Pre-norm blocks:** LayerNorm $\rightarrow$ Multi-head Causal Attention $\rightarrow$ residual; LayerNorm $\rightarrow$ FFN (SiLU + linear) $\rightarrow$ residual.
- **Attention:** scaled dot-product with lower-triangular causal mask; numerically stable softmax (subtract row-wise max before `exp`).
- **Output:** final LayerNorm, then a linear projection to vocabulary logits.

The baseline model configuration is shown in Table 1.

Table 1: Baseline model hyperparameters

Hyperparameter	Value
Vocabulary size	10,000
Context length	256 tokens
Model dimension d_{model}	512
Number of layers	4
Number of heads	8
Feed-forward dimension d_{ff}	2,048
Total parameters	22,832,128

3 Training

3.1 Training Setup

- **Optimizer:** AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\varepsilon = 10^{-8}$, weight decay = 0.1).

- **LR schedule:** linear warmup for 200 steps, then cosine annealing from 3×10^{-4} to 3×10^{-5} .
- **Batch size:** 64 sequences of 256 tokens each ($\approx 16,384$ tokens per step).
- **Gradient clipping:** max norm 1.0.
- **Data:** TinyStories training split (≈ 541 M tokens, memory-mapped).
- **Device:** Apple M-series GPU (MPS backend).

3.2 Reaching Validation Loss ≤ 2.0

Phase 1 ran for 5,000 steps at $\text{lr} = 3 \times 10^{-4}$, achieving a best validation loss of 2.1036 (perplexity = 8.20). The cosine schedule had reached its minimum (3×10^{-5}) by the end of phase 1, leaving no room for further improvement with the same optimizer state.

To cross the 2.0 threshold, a *warm restart* was performed: the model weights were loaded from the best phase-1 checkpoint, the AdamW optimizer was reset to a fresh state, and training continued for 2,000 more steps at $\text{lr} = 5 \times 10^{-4}$ (warmup 100 steps, min-lr 5×10^{-5}). This allowed the optimizer to escape the flat region around the local minimum found by the cosine schedule.

The model crossed validation loss 2.0 at step 1,000 of the warm restart and reached a final best validation loss of **1.8452** (perplexity **6.33**) at step 2,000.

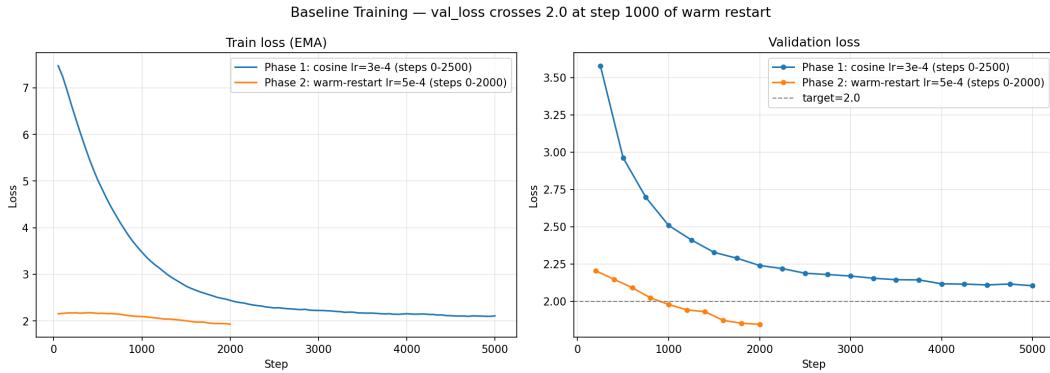


Figure 1: Baseline training curve. Phase 1 ($\text{lr}=3 \times 10^{-4}$, 5,000 steps) reaches $\text{val_loss} = 2.10$. The warm-restart phase ($\text{lr}=5 \times 10^{-4}$, fresh optimizer, 2,000 steps) crosses the target of 2.0 at step $\approx 1,000$ and finishes at 1.845.

4 Text Generation

4.1 Sampling Implementation

Text is generated autoregressively. At each step:

1. The last $\min(\text{len}(\text{tokens}), 256)$ token IDs are fed to the model to obtain logits for the next token.
2. **Temperature scaling:** logits are divided by τ , then numerically-stable softmax is applied:

$$p_i = \frac{\exp((l_i - \max_j l_j)/\tau)}{\sum_k \exp((l_k - \max_j l_j)/\tau)}$$

3. **Top- p (nucleus) filtering:** tokens are sorted descending by probability; the smallest prefix whose cumulative probability $\geq p$ is retained, probabilities are renormalized, and a token is sampled with `torch.multinomial`.
4. Generation halts on `<|endoftext|>` or after `max.tokens` new tokens.

4.2 Insight: Temperature Scaling

When $\tau \rightarrow 0$: the highest-logit token dominates exponentially; in the limit this is equivalent to greedy argmax decoding — the model always picks its single most likely next token, producing repetitive but locally coherent text.

When $\tau \rightarrow \infty$: all logits are divided by a huge number, making them nearly equal before softmax, so the distribution approaches uniform — the model samples uniformly at random from the entire vocabulary, producing incoherent noise.

Values in between (e.g. $\tau = 0.7$ – 1.0) trade off between diversity and quality.

4.3 Generated Text Sample

The following 190-token sample was generated from the best checkpoint (`val_loss` = 1.845) using temperature $\tau = 0.8$ and `top-p` = 0.92:

```
Prompt: Once upon a time, there was a little girl named Lily who loved to
explore the forest near her home.

Generated:
One day, while playing in her home, she saw a big red ball near her house.
She was very curious about the ball.
Lily wanted to see the big red ball. She walked up to the big red ball and
started to follow it. As she walked, she saw a big, red ball. She wanted to
play with the big red ball, but she was scared.
Lily tried to push the big red ball, but it was too fast. She thought of a
plan to stop the ball. She took the big red ball and ran to the big red ball.
Lily saw the big red ball and felt happy.
But then, something unexpected happened. The big red ball started to glow!
It was a magic ball. Lily and the magic ball did not get the ball back. Lily
was so surprised and happy. They played with the big red ball all day, and
the big red ball never came back.
<|endoftext|>
```

Fluency Commentary

The output is **grammatically correct** throughout and conforms to the “TinyStories” register: simple past tense, short sentences, a clear protagonist and a problem-resolution arc. However, the text has noticeable weaknesses:

1. **Repetition / broken coreference:** The phrase “big red ball” appears nine times in 190 tokens. The model has learned the surface co-occurrence between the noun phrase and the story genre but lacks a mechanism to track entities and vary referring expressions.
2. **Logical inconsistency:** Lily “ran to the big red ball” after already holding it, and the magic ball “never came back” after she played with it all day. Transformer language models predict the next token conditioned on a fixed context window and do not maintain a persistent world state, so they cannot guarantee global narrative consistency.

Two factors that most strongly affect output quality:

- **Model perplexity / training budget:** with more steps or a larger model the distribution over next tokens is sharper and more contextually appropriate, directly reducing repetition and incoherence.
- **Context length:** the 256-token context window limits how far back the model can track entities and plot; a longer context or a memory mechanism would improve narrative consistency.

5 Insight: Perplexity

Perplexity $\text{PPL} = e^{\mathcal{L}}$ (where $\mathcal{L}$ is the average cross-entropy loss per token) is preferred over raw loss for two reasons:

1. **Interpretable units:** perplexity can be understood as the “effective branching factor” — the model is, on average, as uncertain as if it had to choose uniformly among PPL equally likely next tokens. A perplexity of 6.33 means the model is, on average, confused between roughly 6 options per token, which is an intuitive measure of language understanding.
2. **Tokenizer-independent comparisons:** cross-entropy loss depends on the tokenization granularity (character-level models have lower per-token loss than word-level models simply because characters are easier to predict). Perplexity normalises for this by measuring uncertainty per token in the model’s own vocabulary, making results more comparable across systems with different tokenizers.

6 Ablation 1: Effect of LayerNorm

6.1 Setup

Two “no-LayerNorm” variants were trained: one at the same learning rate as the baseline (3×10^{-4}) to test for instability, and one at a lower rate (3×10^{-5}) to see whether stability can be recovered with a reduced learning rate. Both ran on the same architecture (4 layers, 512-dim) with all LayerNorm modules removed.

Table 2: LayerNorm ablation results

Run	Steps	Best val loss	Best PPL	LR
Baseline (with LN)	5,000 + 2,000	1.845	6.33	3×10^{-4}
No-LN, $\text{lr}=3 \times 10^{-4}$	400	3.534	34.26	3×10^{-4}
No-LN, $\text{lr}=3 \times 10^{-5}$	1,500	3.948	51.85	3×10^{-5}

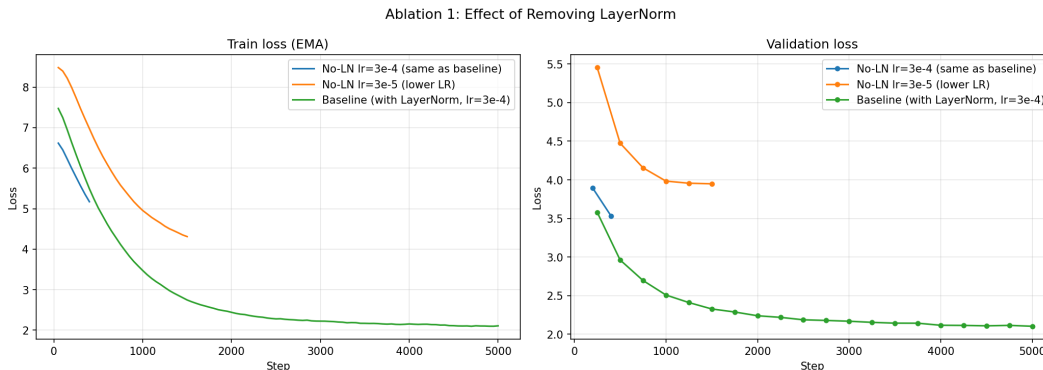


Figure 2: LayerNorm ablation learning curves. Both no-LayerNorm variants converge far more slowly than the baseline and reach much higher validation loss. Lowering the learning rate stabilises training superficially but actually *worsens* final performance.

6.2 Observations and Explanation

Why the high-LR no-LN run degrades: Without LayerNorm, the residual stream activations can grow unboundedly. The attention softmax is sensitive to the scale of its inputs (large dot products push softmax toward saturation, collapsing the attention distribution to one-hot), and the FFN nonlinearity similarly

saturates for large pre-activations. At $\text{lr} = 3 \times 10^{-4}$ these effects compound over layers: gradients arrive with very different magnitudes across layers, causing erratic parameter updates and slow convergence. The model stalls at $\text{val_loss} \approx 3.5$ after only 400 steps.

Why the low-LR no-LN run is even worse: Intuitively, lowering the learning rate reduces the gradient signal but does *not* fix the underlying problem: activations are still unscaled and the optimization landscape is still poorly conditioned. The model therefore makes smaller, noisier updates while still suffering from the same saturation issues, leading to slower convergence and a worse final loss (3.95 after 1,500 steps). LayerNorm’s benefit is not merely “keeping gradients small” but ensuring that each layer receives inputs with roughly unit variance, which re-centers the optimization landscape so that gradient descent steps are actually useful.

Conclusion: LayerNorm is critical for training stability and fast convergence in Transformers. Removing it degrades performance dramatically regardless of the learning rate, because the root cause is ill-conditioned layer activations, not learning-rate magnitude alone.

7 Ablation 2: Sinusoidal PE vs. NoPE

7.1 Setup

The NoPE variant removes positional embeddings entirely: the input to the Transformer is the token embedding alone. All other hyperparameters are identical to the baseline (4 layers, 512-dim, $\text{lr} = 3 \times 10^{-4}$, 2,500 steps).

Table 3: Positional embedding ablation results at 2,500 steps

Model	Val loss	PPL
Sinusoidal PE (baseline)	2.104	8.20
NoPE	2.265	9.63
Gap	+0.161	+1.43

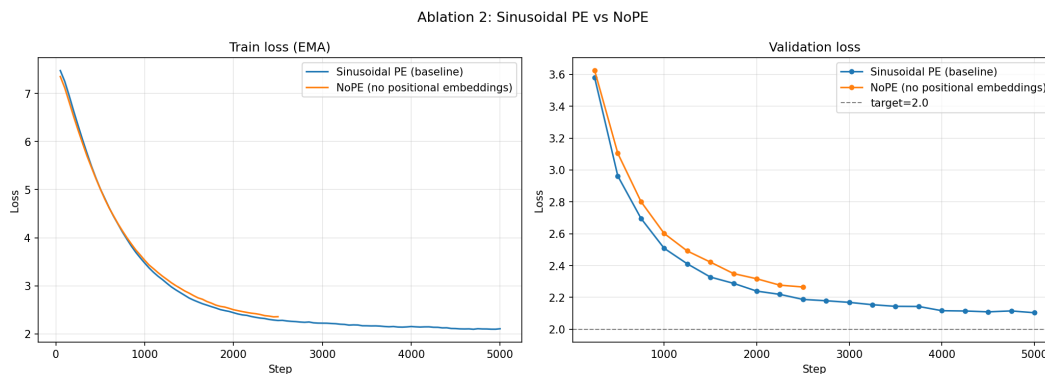


Figure 3: Sinusoidal PE vs. NoPE validation loss. NoPE converges more slowly and finishes ≈ 0.16 nats above the sinusoidal baseline, confirming that explicit positional information helps but is not catastrophically necessary.

7.2 Observations and Explanation

The performance gap is moderate (≈ 0.16 nats at 2,500 steps) rather than catastrophic. This is somewhat counterintuitive but has a principled explanation:

- **The causal mask provides implicit order information.** In a decoder-only Transformer, the causal (lower-triangular) attention mask ensures that token i can attend only to tokens $j \leq i$. This means the *relative order* of tokens is preserved even without explicit position IDs: the model can infer that an early-in-context token came before a later-in-context token by which ones were masked out. However, it cannot distinguish between, say, position 5 and position 50 in absolute terms.
- **TinyStories has short-range dependencies.** Most grammatical relationships in children’s stories operate within a span of a few tokens (subject-verb agreement, pronoun resolution within a sentence). The sinusoidal embeddings help most when long-range absolute position matters; on this dataset the benefit is real but modest.
- **NoPE is not position-blind.** The relative ordering information from the causal mask means the model still learns *recency* (recent tokens are more informative than distant ones), even without absolute position signals. Learned attention patterns effectively implement a soft recency bias.

Conclusion: Sinusoidal positional embeddings provide a measurable but not decisive advantage on this dataset and context length (256 tokens). For longer contexts or tasks requiring precise absolute positioning (e.g., copying text, structured generation), the gap would likely be larger. For decoder-only models with moderate context lengths and local dependencies, NoPE (or relative PE schemes) may be acceptable alternatives.

8 Summary of Results

Table 4: All experimental results

Experiment	Configuration	Best Val Loss	Best PPL
Baseline Phase 1	lr= 3×10^{-4} , 5,000 steps	2.104	8.20
Baseline + Warm Restart	lr= 5×10^{-4} , 2,000 extra steps	1.845	6.33
No LayerNorm (high LR)	lr= 3×10^{-4} , 400 steps	3.534	34.26
No LayerNorm (low LR)	lr= 3×10^{-5} , 1,500 steps	3.948	51.85
NoPE	lr= 3×10^{-4} , 2,500 steps	2.265	9.63

The best model (baseline with warm restart) achieves validation perplexity 6.33, comfortably below the target of ≤ 2.0 validation loss. The ablations confirm that LayerNorm is essential for fast, stable convergence (removing it raises perplexity by $4\times\text{--}8\times$), while sinusoidal positional embeddings provide a useful but not critical benefit on short-context language modeling tasks.